

TÀI LIỆU ÔN BẢO VỆ LUẬN VĂN

Kiến thức nền — Đào sâu theo thứ tự ưu tiên

CIES: Kỹ thuật xử lý mất cân bằng ảnh hưởng thế nào tới độ ổn định
của giải thích SHAP trong phát hiện gian lận thẻ tín dụng

Biên soạn từ [reports/study_guide.md](#), đối chiếu trực tiếp với 11 file mã nguồn trong [src/](#)

Cập nhật 2026-10-03

Mục lục & cách dùng

Tài liệu chia 3 **tier** theo mức độ bắt buộc phải hiểu sâu khi bảo vệ. Mỗi mục gồm: **Khái niệm** (định nghĩa cốt lõi) → **Cơ chế** (công thức/logic thật) → **Trích code** (nguyên văn từ source, có file + số dòng) → **Vì sao dùng ở đây** → **Câu hỏi phản biện**. Đọc Tier 1 trước — đó là những gì hội đồng gần như chắc chắn hỏi vì nó là "trái tim" của đề tài.

TIER 1 — Bắt buộc hiểu sâu (core mechanism)

1. Bootstrap resampling — nền tảng đo độ ổn định
2. Quy trình 5 bước của 1 run CIES
3. Rank-weighted distance & công thức CIES score
4. 4 họ SHAP Explainer: Tree/Linear (exact) vs Deep/Kernel (approx) — và dữ liệu nền
5. Time-based split & Customer-identity leakage
6. PR-AUC vs ROC-AUC trong bài toán cực mất cân bằng

TIER 2 — Nên hiểu vững (cơ chế kỹ thuật của pipeline)

7. 5 kỹ thuật xử lý mất cân bằng — cơ chế nội suy & trọng số lớp
8. Ablation one-hot phân số (SNAP_SYNTHETIC_ONEHOT)
9. Target Encoding out-of-fold + StratifiedGroupKFold (leakage khi bootstrap)
10. Optuna: TPE Sampler, MedianPruner, time-series validation folds
11. Kiến trúc 5 model & vì sao mỗi model cần xử lý riêng

TIER 3 — Biết cơ chế cơ bản (hạ tầng, ít bị hỏi sâu)

12. Cô lập subprocess (spawn vs fork) & xung đột OpenMP
13. Ghi file JSON an toàn & cache resample có fingerprint
14. Hạ tầng reproducibility & pattern "kind" dispatch

TIER 1 — Bắt buộc hiểu sâu

Đây là cơ chế cốt lõi tạo ra con số CIES. Không hiểu được các mục này thì không thể trả lời bất kỳ câu phản biện nào về "đề tài đo cái gì, đo bằng cách nào".

1 Bootstrap resampling — nền tảng đo độ ổn định

KHÁI NIỆM

Bootstrap là lấy mẫu **có hoàn lại** (with replacement) từ 1 tập dữ liệu gốc, tạo ra 1 "phiên bản" mới cùng cỡ nhưng thành phần dòng hơi khác (một số dòng xuất hiện nhiều lần, một số dòng vắng mặt). Đây là kỹ thuật kinh điển của Efron (1979) để ước lượng độ biến thiên (variance) của 1 estimator khi không có nhiều tập dữ liệu độc lập thật.

CƠ CHẾ TRONG ĐỒ ÁN

Ý tưởng cốt lõi: nếu 1 giải thích SHAP là "ổn định", thì train lại mô hình trên nhiều bản bootstrap khác nhau của cùng 1 tập train phải cho ra **cùng 1 bảng xếp hạng feature**. Bootstrap đóng vai trò "nguồn nhiễu có kiểm soát" — mô phỏng câu hỏi "nếu thu thập lại dữ liệu train hơi khác đi và train lại từ đầu, model có kể lại cùng 1 câu chuyện về feature quan trọng không?".

TRÍCH CODE

```
src/explainability/cies.py#L333 (seed), #L342-345 (bootstrap)
```

```
seed = run_idx # run thứ i dùng seed i (0..19)
base = df_train.reset_index(drop=True)
sampled = base.sample(frac=1.0, replace=True, random_state=seed)
row_groups = sampled.index.to_numpy()
train_resample = sampled.reset_index(drop=True)
```

`frac=1.0, replace=True` = đúng định nghĩa bootstrap (lấy đủ n dòng, có hoàn lại). `random_state=seed` với `seed = run_idx` (0..19) cho 20 lần bootstrap khác nhau nhưng tái lập được.

CÁI GÌ ĐỔI, CÁI GÌ CỐ ĐỊNH GIỮA 20 RUN

Đổi theo run (cùng seed = run_idx)	Cố định ở mọi run
Mẫu bootstrap của tập train; phần ngẫu nhiên của resampler (SMOTE...); phần ngẫu nhiên của model (khởi tạo trọng số ANN, lấy mẫu của RF/XGBoost)	Tập eval (250 mẫu), tham số model (đã tune), kỹ thuật imbalance, dữ liệu nền SHAP (100 dòng gốc), công thức encoding

Vì sao dùng ở đây: CIES đo độ dao động của giải thích khi **train lại từ đầu** — gồm cả dữ liệu train đổi (bootstrap) lẫn tính ngẫu nhiên vốn có của thuật toán. Mọi yếu tố nằm ngoài model (tập eval, tham số, dữ liệu nền SHAP) được cố định, nên dao động đo được thuộc về chính "model × kỹ thuật". 5 kỹ thuật dùng chung seed nên dùng chung 20 mẫu bootstrap — so sánh giữa kỹ thuật là so theo cặp.

Q: Bootstrap có tạo dòng trùng — có ảnh hưởng gì đến các bước sau không?

Có — xem mục 9 (StratifiedGroupKFold): dòng trùng phải bị buộc vào cùng 1 fold khi target-encode, nếu không sẽ rò rỉ nhãn.

Q: Bootstrap có phải nguồn nhiễu duy nhất không?

Không. Seed của run cũng đặt cho resampler và model, nên CIES gộp 2 nguồn: dữ liệu train đổi và tính ngẫu nhiên của thuật toán. Đây là chủ đích — người dùng thật cũng train lại model với seed khác; điều cần cố định là những thứ *ngoài* model.

2 Quy trình 5 bước của 1 run CIES

KHÁI NIỆM

Mỗi run (trong 20 run) không chỉ là "train lại" — mà lặp lại **toàn bộ pipeline từ đầu** trên dữ liệu bootstrap, để không có bước nào bị "đóng băng" gây thiên lệch giả.

CƠ CHẾ — ĐÚNG 5 BƯỚC, KHÔNG ĐƯỢC RÚT GỌN

`src/explainability/cies.py#L269 (run_cies_experiment)`, khớp comment L8-14

0. (trước vòng lặp) chọn 100 dòng nền SHAP cố định từ tập train gốc # L318-323
1. Bootstrap resample train (KHÔNG đổi eval set)
2. Encoding fit lại trên resample này (encode_train, KHÔNG dùng lại encoding cũ)
3. Imbalance handling (apply_imbalance — chỉ trên train đã encode)
4. Train model (build_model + train_model, tham số ĐÃ ĐÓNG BĂNG từ tune.py)
5. SHAP trên tập eval CỐ ĐỊNH (250 mẫu: 50 fraud + 200 hợp lệ), so với dữ liệu nền cố định

Điểm dễ bị hỏi nhất: nếu bước 2 dùng lại encoding đã fit 1 lần từ đầu (không re-fit mỗi run), thì độ "ổn định" đo được sẽ bị đánh giá **quá cao giả tạo** — vì encoding chính là 1 nguồn dao động (target encoding phụ thuộc thống kê của đúng bản bootstrap đó). Tập eval và 100 dòng nền được encode bằng mapping của run đó.

BẢO VỆ CHỐNG CIES GIẢ (SHAP TOÀN 0)

`src/explainability/cies.py#L400-404`

```
if np.allclose(shap_vals, 0.0):  
    raise ValueError("SHAP toàn 0 — model không học được (dự đoán hằng số), bỏ run này")
```

Nếu model dự đoán hằng số, mọi feature có SHAP = 0 → mọi thứ hạng hoà nhau → khoảng cách = 0 → `cies_score = 1.0`. Đây là "ổn định tuyệt đối" GIẢ (không có gì dao động, không phải vì model nhất quán). Guard này loại run đó khỏi phép đo. Thực tế: cả 1.000 run (25 tổ hợp × 20 run × 2 dataset) đều thành công.

Vì sao dùng ở đây: đây là điểm phân biệt CIES với việc "chỉ so 2 bảng SHAP là xong" — độ nghiêm ngặt của quy trình (re-encode, nền cố định, guard chống giả) là thứ khiến con số có ý nghĩa.

3 Rank-weighted distance & công thức CIES score

KHÁI NIỆM

CIES cần 1 hàm đo "khoảng cách" giữa 2 bảng xếp hạng feature, nhưng không đối xử mọi feature như nhau: đảo hạng ở **feature quan trọng nhất** (rank 1, 2...) phải bị phạt nặng hơn đảo hạng ở feature ít quan trọng (rank 18, 19...) — vì đảo hạng ở top mới thực sự đổi "câu chuyện" mà model kể.

CƠ CHẾ — CÔNG THỨC ĐẦY ĐỦ

`src/explainability/cies.py#L44-83 (compute_rank_weighted_distance)`

```
weights = 1.0 / np.minimum(ranks_a, ranks_b) # trọng số theo hạng TỐT NHẤT giữa 2 lần
weights = weights / weights.sum()           # chuẩn hoá tổng = 1
rank_diff = np.abs(ranks_a - ranks_b)
normalized_diff = rank_diff / (n - 1)       # chuẩn hoá về [0, 1]
distance = np.sum(weights * normalized_diff)
```

Điểm tinh tế: trọng số dùng `min(rank_a, rank_b) - 1` feature từng lọt top ở ít nhất 1 trong 2 lần chạy thì thay đổi của nó đã bị phạt nặng. Comment trong code ghi lại 1 lỗi cũ: trọng số từng gán theo **vị trí cột** (index i) thay vì hạng thực tế — làm kết quả phụ thuộc thứ tự cột feature.

TỪ KHOẢNG CÁCH SANG CIES SCORE

`src/explainability/cies.py#L148-210 (compute_stability_metric)`

```
rankings = [shap_to_ranks(sv) for sv in shap_values_runs] # 20 runs → 20 rankings
# so mọi CẤP trong 20 run: C(20,2) = 190 cặp
for i in range(n_runs):
    for j in range(i+1, n_runs):
        distances.append(compute_rank_weighted_distance(rankings[i], rankings[j]))
cies_score = 1.0 - mean(distances) # 1 = ổn định tuyệt đối
```

`shap_to_ranks` (L101-115) xếp hạng feature theo `mean(|SHAP|)` qua các mẫu eval — dùng `scipy.stats.rankdata(method="average")` để feature hoà điểm chia đều hạng trung bình, tránh chênh lệch giả chỉ do thứ tự cột (comment L90-94).

MỐC ĐỂ ĐỌC 1 CON SỐ CIES (ĐO BẰNG CHÍNH HÀM TRÊN, 21 FEATURE)

Tình huống	CIES
Thứ hạng hoàn toàn ngẫu nhiên giữa các run	≈ 0,56 (không phải 0 — vì khoảng cách chuẩn hoá theo n-1)
Chỉ đổi chỗ top-1 với top-2	0,976
Chỉ đổi chỗ 2 feature cuối bảng	0,999

Vậy CIES 0,91 nghĩa là trung bình các run đảo hạng nhiều hơn 1 lần đổi chỗ top-2.

MA TRẬN ĐỒNG THUẬN (KHÁC CIES, DỄ NHẦM)

`src/explainability/cies.py#L118-145 (compute_condition_agreement_matrix)`

Dùng CÙNG công thức rank-weighted distance nhưng so **giữa các điều kiện khác nhau** (vd. 5 kỹ thuật imbalance của cùng 1 model) — CIES gốc là so **giữa các lần bootstrap của CÙNG 1 điều kiện**.

Vì sao thiết kế vậy: Spearman thông thường đối xử mọi hạng như nhau, không phân biệt "top-1 đảo với top-2" (ngghiêm trọng) và "hạng 17 đảo với hạng 18" (không đáng kể).

Q: Vì sao cần cả Spearman lẫn CIES?

Spearman (L191-196, cùng hàm `compute_stability_metric`) là đối chiếu độc lập. Khi 2 chỉ số lệch nhau (vd. ANN Sparkov: CIES 0,87–0,92, Spearman 0,72–0,86), nghĩa là top feature giữ vị trí còn phần đuôi xáo trộn — CIES phạt nhẹ phần đuôi, Spearman thì không.

4 họ SHAP Explainer: exact vs approximate — và dữ liệu nền

KHÁI NIỆM

SHAP gán mỗi feature 1 "đóng góp" vào dự đoán, dựa trên Shapley value (lý thuyết trò chơi hợp tác: trung bình đóng góp biên qua mọi thứ tự feature). Tính chính xác đòi hỏi duyệt 2^n tổ hợp feature — không khả thi, nên mỗi loại model có 1 thuật toán riêng.

CƠ CHẾ — MAPPING BẮT BUỘC THEO MODEL

[src/explainability/shap_utils.py#L23-33 \(EXPLAINER_MAP\)](#)

Model	Explainer	Loại	Cần dữ liệu nền?
Logistic Regression	LinearExplainer	Exact (công thức đóng)	Có
RF / XGBoost / CatBoost	TreeExplainer	Exact (khai thác cấu trúc cây, Lundberg 2020)	Không — dùng số mẫu đi qua từng nhánh, ghi lúc train
ANN	DeepExplainer	Xấp xỉ, tắt định (DeepLIFT)	Có
ANN (fallback)	KernelExplainer	Xấp xỉ, có nhiều (lấy mẫu Monte Carlo)	Có

DỮ LIỆU NỀN (BACKGROUND) — VÌ SAO PHẢI CỐ ĐỊNH GIỮA CÁC RUN

SHAP luôn là đóng góp **so với 1 mốc**. Với LR: $SHAP_j = \text{hệ số}_j \times (x_j - \text{trung bình nền}_j)$. Nếu nền đổi theo run, mốc dịch chuyển và thứ hạng đảo dù model không đổi. Ví dụ minh họa: cùng 1 model, nền A cho `amt = 1,40 > hour = 1,20`; nền B (trung bình khác) cho `amt = 1,10 < hour = 1,32` — thứ hạng đảo chỉ vì mốc.

[src/explainability/cies.py#L318-323 \(chọn nền\), #L390-395 \(encode nền theo run\)](#)

```
background_raw = df_train.sample(n=min(100, len(df_train)), random_state=SEED) # 1 lần, trước vòng lặp
...
encoded_bg = encode_test(background_raw, encoding_maps, ...) # mỗi run: encode bằng mapping của run
X_background = encoded_bg[train_feature_cols].values
```

Số đo (giữ nguyên các model đã train, chỉ đổi nền; Sparkov): nền lấy từ mẫu bootstrap của từng run cho CIES thấp hơn nền cố định — LR $\times$ SMOTE 0,912 so với 0,924; ANN $\times$ SMOTE 0,838 so với 0,867. Feature bị ảnh hưởng nhất: các cột one-hot `category_*` hiếm (trung bình trên 100 dòng của 1 category ~3% dao động mạnh) và `amt` (đuôi dài). ULB (feature PCA liên tục) gần như không đổi. TreeExplainer không dùng nền nên model cây không bị ảnh hưởng.

VÌ SAO KHÔNG DÙNG KERNEL CHO ANN — ĐIỂM HAY BỊ HỎI

[src/explainability/shap_utils.py#L28-31 \(comment\)](#)

```
"DeepExplainer thay KernelExplainer: đo trên ANN thật, Kernel chạy lặp 2 lần cùng
model chỉ khớp nhau Spearman ~0.85 (nhiều lấy mẫu riêng — sẽ làm điểm CIES của ANN
thấp GIẢ TẠO vì CIES đo đúng độ dao động thứ hạng), chậm hơn ~5x, còn Deep
tắt định, cộng dồn đúng (sai số 0) và cùng đơn vị log-odds với LR/XGBoost."
```

Kernel xấp xỉ Shapley bằng lấy mẫu ngẫu nhiên — 2 lần gọi trên cùng model, cùng input vẫn cho kết quả hơi khác. Nếu dùng Kernel, phần "bất ổn" đo được sẽ trộn giữa (a) dao động thật của model — thứ CIES muốn đo — và (b) nhiễu của chính explainer. Cùng logic với việc cố định dữ liệu nền: loại mọi nguồn dao động nằm ngoài model.

TRÍCH CODE — LOGIC TỰ LUI (FALLBACK) KHI DEEPEXPLAINER LỖI

`src/explainability/shap_utils.py#L227-234`

```
try:
    explainer = shap.DeepExplainer(ann, X_bg_tensor)
    shap_values = explainer.shap_values(X_eval_tensor)
except Exception as e:
    logger.warning(f"DeepExplainer lỗi (...) — lùi về KernelExplainer.")
    return _compute_shap_kernel(model, X_eval, X_background), "kernel"
```

Tên explainer thực sự đã dùng được ghi vào log mỗi run (`cies.py#L419`: `"explainer": explainer_used`). Cả 200 run ANN đều dùng Deep — fallback chưa từng kích hoạt.

SHAP GIẢI THÍCH TRONG KHÔNG GIAN NÀO?

`shap_utils.py#L86-115` (`_compute_shap_linear`), `L200-243` (`_compute_shap_deep`)

LR: SHAP tính trên input đã **StandardScaler** (cùng scaler lúc train) — nếu quên, LinearExplainer đọc hệ số fit trên không gian scale nhưng nhận input ở không gian gốc → SHAP sai hoàn toàn (comment L92-95). ANN: SHAP giải thích **logit** (trước sigmoid) — cùng thang log-odds với LR/XGBoost.

Vì sao dùng ở đây: để tài so độ ổn định SHAP *giữa các model khác nhau* — nếu 1 model có thêm nguồn dao động từ explainer (nhiều Kernel) hay từ mốc so sánh (nền đổi theo run), kết luận "model X ổn định hơn model Y" có thể chỉ phản ánh cách tính SHAP, không phải bản chất model.

Q: TreeExplainer là exact, DeepExplainer là approximate — điều này có làm lệch so sánh CIES giữa cây và ANN không?

Có thể. Deep là tắt định nên không thêm nhiễu giữa các run, nhưng nó là 1 phương pháp gần đúng khác (DeepLIFT) — không có cách loại bỏ yếu tố này trong thiết kế hiện tại. Cần thừa nhận thẳng là giới hạn khi so CIES giữa model cây và ANN; so sánh **giữa các kỹ thuật trong cùng 1 model** thì không bị ảnh hưởng.

5 Time-based split & Customer-identity leakage

KHÁI NIỆM

Data leakage là khi thông tin không có sẵn tại "thời điểm dự đoán thật" lọt vào train, làm hiệu năng đo được cao giả tạo. Có 2 dạng riêng biệt trong đồ án, dễ bị hỏi lẫn lộn: **temporal leakage** (rò rỉ tương lai) và **identity leakage** (rò rỉ danh tính).

CƠ CHẾ 1 — TEMPORAL LEAKAGE

```
src/data/preprocess.py#L28 (sort_values theo TIMESTAMP_COL)
```

```
data = df.sort_values(TIMESTAMP_COL, kind="mergesort").reset_index(drop=True)
```

Nếu chia ngẫu nhiên, các dòng của **cùng 1 đợt hack thẻ** (fraud thường xảy ra theo chuỗi) rơi vào cả train lẫn test → model "nhớ" đúng đợt hack đó thay vì học pattern tổng quát. Sparkov được phát hành sẵn theo thời gian (fraudTrain.csv → fraudTest.csv); ULB lấy 20% cuối theo Time.

CƠ CHẾ 2 — IDENTITY LEAKAGE (TÍNH VI HƠN)

```
src/config.py#L67-72
```

```
CUSTOMER_IDENTITY_COLS = ["lat", "long", "city", "state", "job", "city_pop",  
                           "merch_lat", "merch_long"]  
# 98,6% cặp (lat, long) và 90,1% giá trị city_pop chỉ thuộc 1 thẻ,  
# 92,2% city chỉ có 1 thẻ; merch_lat/long luôn nằm trong ~1,4° quanh nhà khách.
```

Các cột này trông như feature hợp lệ nhưng gần như chỉ ra đúng 1 khách hàng → model học "khách nào từng bị hack" thay vì "giao dịch nào đáng ngờ".

SỐ ĐO (XGBOOST + CLASS_WEIGHTING, 1 LẦN CHIA, 1 SEED)

Bộ feature	Chia ngẫu nhiên theo dòng	Chia theo thời gian
Giữ cả cột định danh	0,932 (đẹp giả — nhớ khách)	0,240 (sập — khách bị hack ở test khác ở train)
Bỏ cột định danh (thiết kế hiện tại)	—	0,882 (pipeline thật: 0,883)

Đọc bảng: con số 0,932 phồng lên nhờ **cả hai** loại rò rỉ cùng lúc; chỉ chia theo thời gian mà giữ định danh thì PR-AUC sập còn 0,240; bỏ định danh thì model học được tín hiệu thật (0,882).

KIỂM CHỨNG "BỎ KHÔNG MẤT TÍN HIỆU THẬT"

Khoảng cách khách hàng—cửa hàng của giao dịch fraud và hợp lệ gần như trùng nhau (76,3 km so với 76,1 km) — "giao dịch xa nhà" không phân biệt được fraud; thứ bị mất khi bỏ các cột là khả năng tra cứu ngược danh tính.

Vì sao dùng ở đây: đây là 2 lớp phòng thủ chống 2 cơ chế ảo tưởng hiệu năng khác nhau — làm 1 trong 2 thì PR-AUC vẫn sai lệch.

Q: Sao phát hiện được leakage?

Dấu hiệu đầu tiên: chuyển sang chia theo thời gian thì PR-AUC sập từ 0,93 xuống 0,24 — model đã dựa vào thứ không khái quát sang tương lai. Sau đó mới định lượng bằng tỷ lệ giá trị chỉ thuộc 1 thẻ.

6 PR-AUC vs ROC-AUC trong bài toán cực mất cân bằng

KHÁI NIỆM

ROC-AUC: diện tích dưới đường cong TPR (Recall) theo FPR. **PR-AUC** (= Average Precision): diện tích dưới đường cong Precision theo Recall. Cả 2 không phụ thuộc 1 ngưỡng cụ thể nhưng phản ứng rất khác nhau khi lớp dương cực hiếm.

CƠ CHẾ TOÁN HỌC – VÌ SAO ROC-AUC "PHÓNG ĐẠI"

$FPR = FP / (FP + TN)$. Khi TN cực lớn (Sparkov test: 555.719 dòng, chỉ 2.145 fraud), FP tăng gấp đôi vẫn rất nhỏ so với TN $\rightarrow$ FPR gần như không đổi $\rightarrow$ ROC-AUC khó phân biệt model tốt/tệ (FDB báo ROC-AUC 0,995–0,998 cho mọi model). $Precision = TP / (TP + FP)$ nhạy với FP tuyệt đối.

`src/evaluation/metrics.py#L49–53`

```
metrics["pr_auc"] = average_precision_score(y_true, y_proba)    # METRIC CHÍNH
metrics["roc_auc"] = roc_auc_score(y_true, y_proba)             # chỉ tham khảo
```

`average_precision_score` = tổng có trọng số của Precision tại mỗi mức recall đạt được — không nội suy tuyến tính giữa 2 điểm (tránh lạc quan quá mức).

Vì sao dùng ở đây: Saito & Rehmsmeier (2015) chỉ ra ROC-AUC gây hiểu lầm khi lớp dương rất hiếm. PR-AUC còn không phụ thuộc ngưỡng: resample/class weighting đẩy xác suất dự đoán lên (vd. class weighting báo nhầm 4.606 giao dịch với XGBoost ở ngưỡng 0,5) nhưng ít đổi thứ tự xếp hạng — F1 ở ngưỡng 0,5 thì bị ảnh hưởng mạnh.

TIER 2 — Nên hiểu vững

Cơ chế kỹ thuật đứng sau từng bước của pipeline. Hội đồng có thể hỏi "vì sao chọn cách này" — cần hiểu đủ để giải thích, không cần thuộc lòng từng dòng code.

7 5 kỹ thuật xử lý mất cân bằng — cơ chế thật

KHÁI NIỆM CHUNG

2 nhóm khác cơ chế: **4 kỹ thuật resampling** (SMOTE, SMOTE-ENN, ADASYN, Borderline-SMOTE) sinh/xoá dòng dữ liệu; **class weighting** không đổi dữ liệu, chỉ đổi *trọng số trong hàm mất mát*.

```
src/imbalance/resamplers.py#L48-54 (get_resampler)
```

```
resamplers = {
    "smote": SMOTE(random_state=seed),
    "smote_enn": SMOTEENN(random_state=seed),
    "adasyn": ADASYN(random_state=seed),
    "borderline_smote": BorderlineSMOTE(random_state=seed),
    "class_weighting": None,
}
```

- **SMOTE**: với mỗi điểm fraud, chọn ngẫu nhiên 1 trong $k=5$ láng giềng cùng lớp, nội suy tuyến tính (điểm giả = gốc + $t \times (\text{láng giềng} - \text{gốc})$, $t \in [0,1]$) → cân bằng về 1:1.
- **ADASYN**: như SMOTE nhưng sinh nhiều điểm hơn ở vùng fraud khó phân loại.
- **Borderline-SMOTE**: chỉ nội suy quanh các điểm fraud nằm sát ranh giới (xác định qua tỷ lệ láng giềng khác lớp).
- **SMOTE-ENN**: SMOTE trước, rồi Edited Nearest Neighbours xoá các điểm (cả 2 lớp) bị đa số láng giềng khác lớp bao quanh.

CLASS WEIGHTING (KHÔNG SINH DÒNG MỚI)

```
src/imbalance/resamplers.py#L65-79 (get_class_weights)
```

```
weights = compute_class_weight("balanced", classes=classes, y=y)
# công thức sklearn:  $w_c = n_{\text{samples}} / (n_{\text{classes}} * n_{\text{samples}_c})$ 
```

Trên Sparkov (~0,58% fraud): trọng số lớp fraud ≈ 86 , lớp hợp lệ $\approx 0,5$ — phân loại sai 1 fraud bị phạt gấp ~172 lần. Mỗi thư viện nhận trọng số theo cách khác nhau:

Model	Cách nhận trọng số	File#Line
LR / RF	<code>class_weight={0: w0, 1: w1}</code> trực tiếp vào sklearn	train.py#L166, L185
XGBoost	<code>scale_pos_weight = w1/w0</code> (tỷ lệ, không phải dict)	train.py#L211-215
CatBoost	<code>class_weights=[w0, w1]</code> (list theo thứ tự lớp)	train.py#L237-240
ANN	<code>pos_weight</code> trong BCEWithLogitsLoss	train.py#L358-366

Comment ở train.py#L206-210 ghi lại 1 lỗi đã sửa: `scale_pos_weight` từng bị tính **ngược** ($w_0/w_1 \approx 0,0017$ thay vì ≈ 577 trên ULB), khiến fraud bị GIẢM trọng số — PR-AUC XGBoost $\times$ class_weighting trên ULB tụt $0,877 \rightarrow 0,704$.

Vì sao dùng ở đây: `get_class_weights()` trả 1 dict chuẩn `{0: w0, 1: w1}` và `build_model()` tự chuyển cho đúng từng thư viện — tránh lặp logic tính toán ở 4 nơi.

Q: Kết quả chính về kỹ thuật là gì?

Borderline-SMOTE cho CIES thấp nhất ở 9/10 cặp model × dataset. Chênh lệch nhỏ (0,0005–0,011 với kỹ thuật kế tiếp trên Sparkov): so với sai số jackknife (0,002–0,006), ở cả 9 cặp nó thua kỹ thuật tốt nhất hơn 2 lần sai số, nhưng 4/9 cặp chưa tách được khỏi kỹ thuật kế tiếp. Model giải thích phần lớn chênh lệch CIES (Sparkov 86%, kỹ thuật 7%).

8 Ablation one-hot phân số (SNAP_SYNTHETIC_ONEHOT)

KHÁI NIỆM

SMOTE/ADASYN/Borderline-SMOTE nội suy tuyến tính trên **mọi cột**, kể cả cột one-hot (gender, category) — vốn chỉ nhận 0 hoặc 1. Nội suy giữa 2 dòng khác category tạo giá trị phân số (vd. 0,7/0,3), điều không có ở dữ liệu thật.

```
src/config.py#L96–107, src/imbalance/resamplers.py#L98–112 (snap_onehot_groups)
```

```
SNAP_SYNTHETIC_ONEHOT = False # mặc định: SMOTE thuần (one-hot phân số), như literature
# True: ép mỗi nhóm one-hot của MỖI dòng về đúng 1 category (argmax) —
# dòng thật vốn đã hợp lệ nên no-op, chỉ dòng tổng hợp bị thay đổi.
```

Hàm áp argmax cho mọi dòng — dòng thật không đổi gì, chỉ dòng có giá trị phân số bị "chụp" về 1 category. Không cần biết dòng nào là tổng hợp (SMOTE-ENN còn xóa dòng).

KẾT QUẢ ABLATION (XGBOOST + SMOTE, 1 SEED)

Chế độ	PR-AUC (mẫu 300k)	CIES (mẫu 100k)
False (one-hot phân số, mặc định)	0,887	0,957
True (ép hợp lệ)	0,766	0,959

Giả thuyết: giá trị phân số vô tình **cô lập dòng tổng hợp khỏi vùng dữ liệu thật**, giúp model tách biên dễ hơn. CIES gần như không đổi giữa 2 chế độ.

Vì sao mặc định False: để so sánh được với cách làm phổ biến (SMOTE chuẩn, không làm sạch one-hot); dòng tổng hợp chỉ nằm trong train — mọi đánh giá và giải thích đều tính trên giao dịch thật.

9 Target Encoding out-of-fold & leakage khi bootstrap

KHÁI NIỆM

Target encoding thay 1 category (vd. merchant, 693 giá trị) bằng tỷ lệ fraud của category đó. Rủi ro: nếu thống kê tính từ *chính dòng đang encode*, nhãn của dòng đó "rò" vào feature của chính nó.

CƠ CHẾ — OUT-OF-FOLD (OOF) + SMOOTHING

`src/data/encoding.py#L77-90 (stratified_kfold_target_encode)`

```
for train_idx, val_idx in splits:
    # StratifiedKFold, 5 fold
    fold_train = df_reset.iloc[train_idx]
    fold_mean = fold_train[target_col].mean() # prior CHỈ từ fold train
    stats = fold_train.groupby(col)[target_col].agg(["mean", "count"])
    smoothed = (stats["count"]*stats["mean"] + smoothing*fold_mean) / (stats["count"]+smoothing)
    encoded.iloc[val_idx] = df_reset[col].iloc[val_idx].map(smoothed).fillna(fold_mean)
```

Mỗi dòng được encode bằng thống kê từ 4 fold còn lại. **Smoothing = 10** ~ thêm 10 giao dịch "ảo" mang tỷ lệ fraud chung: merchant 2 giao dịch / 1 fraud → 50% thô thành $\approx 8,8\%$; merchant 2.000 giao dịch / 20 fraud → 1% gần như giữ nguyên.

CHI TIẾT ĐÃ SỬA (ĐÁNG NHẮC KHI BỊ HỎI "ĐÃ KIỂM TRA RÒ RÌ CHƯA")

`encoding.py#L79-81 (comment)`

Prior làm mịn từng tính trên toàn bộ train → nhãn của chính các dòng validation lọt vào prior (rò rỉ nhỏ nhưng có thật). Nay prior chỉ tính từ fold train.

VẤN ĐỀ RIÊNG KHI KẾT HỢP VỚI BOOTSTRAP: STRATIFIEDGROUPKFOLD

`encoding.py#L53-56 (docstring), L64-73`

Bootstrap tạo dòng trùng. Với StratifiedKFold thường, 3 bản sao của 1 dòng có thể rơi vào 3 fold khác nhau — bản ở fold validation được encode bằng thống kê chứa nhãn của 2 bản kia, tức chính nhãn của nó. Giải pháp: `groups = row_groups (id dòng gốc) + StratifiedGroupKFold` — mọi bản sao luôn nằm **cùng 1 fold**.

```
skf = StratifiedGroupKFold(n_splits=n_splits, shuffle=True, random_state=random_state)
splits = skf.split(df_reset, df_reset[target_col], groups=np.asarray(groups))
```

Vì sao dùng ở đây: ví dụ của "rò rỉ bậc 2" — 1 kỹ thuật chống rò rỉ (OOF) cần thêm 1 lớp bảo vệ khi kết hợp với kỹ thuật khác (bootstrap).

Q: Tập test encode thế nào?

Dùng tỷ lệ đã làm mịn tính trên **toàn bộ** train (không K-fold); merchant chưa từng xuất hiện ở train nhận tỷ lệ fraud chung. Không bao giờ fit trên test.

10 Optuna: TPE Sampler, MedianPruner, time-series folds

KHÁI NIỆM

TPE (Tree-structured Parzen Estimator) là Bayesian optimization: mô hình hoá phân phối tham số dẫn tới kết quả tốt vs xấu, rồi đề xuất điểm thử tiếp có khả năng tốt cao — khác grid/random search không học từ lịch sử.

VALIDATION THEO THỜI GIAN, KHÔNG PHẢI K-FOLD THƯỜNG

`src/models/tune.py#L94-104 (time_series_folds)`

```
splitter = TimeSeriesSplit(n_splits=n_splits, test_size=n_samples // (3 * n_splits))
# 1/3 CUỐI train chia thành n_splits khối liên tiếp; mỗi fold train trên MỌI dòng
# đứng TRƯỚC khối validation của nó (cửa sổ mở rộng - expanding window)
```

Fold (Sparkov)	Train	Validation
1	01/2019 → 18/12/2019	18/12/2019 → 18/02/2020
2	01/2019 → 18/02/2020	18/02 → 26/04/2020
3	01/2019 → 26/04/2020	26/04 → 21/06/2020

ENCODE RIÊNG TỪNG FOLD

`tune.py#L107-140 (encode_time_folds)`

```
enc_tr, maps = encode_train(part_tr, ...)    # fit CHỈ trên phần train của fold này
enc_val = encode_test(part_val, maps, ...)    # transform validation bằng mapping đó
```

Encode 1 lần trên toàn train rồi mới cắt fold thì target encoding của dòng train đã "nhìn thấy" nhãn của giai đoạn validation phía sau (look-ahead bias).

MEDIANPRUNER (TIẾT KIỆM COMPUTE)

`tune.py#L218-232`

```
if use_pruning:
    trial.report(float(np.mean(val_pr_aucs)), fold_idx)    # báo cáo tạm sau mỗi fold
    if trial.should_prune():
        raise optuna.TrialPruned()
```

Sau mỗi fold, so PR-AUC tạm thời với median của các trial đã xong — kém rõ thì dừng ngay.

`n_startup_trials` đảm bảo vài trial đầu chạy đủ để có median tham chiếu. 50 trial; mọi lượt tune cách kết quả cuối $\leq 0,005$ PR-AUC từ trial 13.

ĐÓNG BĂNG THAM SỐ — DECOUPLED BASELINE PRE-TUNING

`tune.py#L4-9 (module docstring)`

Tune trên dữ liệu **chưa xử lý mất cân bằng**, rồi dùng chung 1 bộ tham số cho cả 5 kỹ thuật — để chênh lệch giữa kỹ thuật là do kỹ thuật, không do tham số. Nhược điểm: tham số không tối ưu riêng cho từng kỹ thuật.

Vì sao dùng ở đây: cùng nguyên tắc "controlled experiment" — cô lập biến độc lập (kỹ thuật imbalance) khỏi các biến gây nhiễu khác.

11 Kiến trúc 5 model & vì sao mỗi model cần xử lý riêng

KHÁI NIỆM

5 model đại diện 3 họ: tuyến tính (LR), cây tổ hợp (RF, XGBoost, CatBoost), mạng nơ-ron (ANN) — khác nhau về tiền xử lý và cách nhận trọng số lớp (mục 7).

VÌ SAO CÂY KHÔNG CẦN SCALE NHƯNG LR/ANN CẦN

```
train.py#L171-174 (comment), L347-351 (comment trong _train_ann)
```

```
# LR: "feature chưa scale ... chênh lệch scale lớn khiến lbfgs khó hội tụ trong max_iter"
# ANN: "Không scale thì lớp Linear đầu nhận giá trị khổng lồ và train gần như hỏng
#       - đo trên Sparkov: PR-AUC 0.16 (không scale) vs 0.74 (có scale)."
```

Cây chỉ so feature $\leq$ ngưỡng tại mỗi node — bất biến với phép biến đổi đơn điệu (kể cả scaling). LR (lbfgs) và ANN (gradient descent) nhạy với thang đo.

KIẾN TRÚC ANN

```
train.py#L42-63
```

```
Input(21 hoặc 29) → Linear(128) → ReLU → BatchNorm1d → Dropout
                  → Linear(64)  → ReLU → BatchNorm1d → Dropout
                  → Linear(32)  → ReLU
                  → Linear(1)    # logit, KHÔNG có Sigmoid
```

Không có Sigmoid cuối vì dùng BCEWithLogitsLoss (gộp Sigmoid + BCE, ổn định số học hơn).

predict_proba tự áp torch.sigmoid (train.py#L434) khi cần xác suất. Mạng nhỏ (~13 nghìn tham số); tham số tune: lr, dropout, epochs (5–50), batch (1024/2048/4096).

RÀNG BUỘC CATBOOST — KHÔNG DÙNG CƠ CHẾ NỘI TẠI

```
train.py#L4-9, L223; tune.py#L12, L67
```

```
# "CatBoost PHẢI dùng chung bộ feature đã encode sẵn — KHÔNG dùng cơ chế
# Ordered Target Statistics nội tại (KHÔNG truyền cat_features). Nếu CatBoost
# dùng encoding riêng, sự khác biệt CIES giữa CatBoost và các model khác
# không tách bạch được — phá vỡ RQ3."
```

Nếu bật, CatBoost nhận feature khác hẳn 4 model kia — chênh lệch CIES không biết là do "bản chất model" hay do "encoding khác nhau".

Vì sao dùng ở đây: mọi khác biệt giữa 5 model phải đến từ *bản thân thuật toán học*, không phải từ việc mỗi model nhận input khác nhau — cùng tinh thần với mục 10 và với việc cố định dữ liệu nền SHAP (mục 4).

TIER 3 — Biết cơ chế cơ bản

Hạ tầng kỹ thuật hỗ trợ chạy benchmark quy mô lớn. Ít khả năng bị hỏi sâu, nhưng nên biết đủ để không bối rối nếu bị hỏi "vì sao code phức tạp vậy".

12 Cô lập subprocess (spawn) & xung đột OpenMP

KHÁI NIỆM

OpenMP là runtime chạy song song đa luồng dùng bởi nhiều thư viện số học (torch, xgboost đều mang libomp riêng). 2 bản libomp khác nhau cùng nạp vào 1 process có thể xung đột → segfault hoặc treo.

`src/models/train.py#L18-28 (comment), src/utils/isolation.py#L16-26, L99`

```
# torch chỉ import khi build ANN; xgboost/catboost chỉ import khi build đúng model đó
ctx = mp.get_context("spawn") # KHÔNG dùng "fork": fork nhân bản nguyên trạng memory của
                               # process cha; spawn khởi động 1 interpreter HOÀN TOÀN MỚI
```

TIMEOUT VỚI LEO THANG SIGTERM → SIGKILL

`isolation.py#L49-72 (_terminate_hard)`

```
process.terminate() # SIGTERM trước
process.join(timeout=KILL_GRACE_SECONDS)
if process.is_alive():
    process.kill() # SIGKILL nếu SIGTERM không đủ
    process.join(timeout=KILL_GRACE_SECONDS)
```

Lý do cụ thể: 1 job Optuna trên Kaggle "Running" hơn 5 giờ dù đặt `timeout=3600s`, vì `join()` sau `terminate()` chờ vô hạn.

Vì sao dùng ở đây: mỗi tổ hợp chạy trong subprocess riêng — 1 tổ hợp treo/crash không làm mất các tổ hợp đã xong.

13 Ghi file JSON an toàn & cache resample có fingerprint

KHÁI NIỆM

Nhiều tiến trình (vd. 2 tiến trình CIES chạy song song) có thể cùng ghi 1 file kết quả — không kiểm soát thì tiến trình ghi sau sẽ mất phần tiến trình khác vừa thêm (đã xảy ra thật: mất 1 tổ hợp đã xong).

```
src/utils/jsonio.py#L18-56 (_exclusive_lock, update_json)
```

```
with _exclusive_lock(path):
    current = json.load(f) if path.exists() else default()
    new = update(current)
    json.dump(new, tmp_file)
    os.replace(tmp, path)
```

#fcntl.flock — chỉ 1 tiến trình được vào
#đọc → sửa → trả về bản mới
#ghi ra file TẠM trước
#đổi tên = ghi atomic

Read-modify-write trong 1 khoá độc quyền; `os.replace` là phép nguyên tử ở tầng OS — không có trạng thái "file ghi dở".

CACHE RESAMPLE CÓ FINGERPRINT (SHA1)

```
src/imbalance/resamplers.py#L158-165 (_resample_fingerprint)
```

```
for part in (technique, seed, SNAP_SYNTHETIC_ONEHOT, onehot_groups,
             imblearn.__version__, sklearn.__version__, X.shape, X.dtype, y.dtype):
    h.update(repr(part).encode())
h.update(X.tobytes()); h.update(y.tobytes())
```

Kết quả resample cho benchmark (SMOTE-ENN trên toàn train Sparkov mất 145 phút) được cache thành `.parquet`, chỉ dùng lại nếu dấu vân tay khớp — dữ liệu, kỹ thuật, seed hay version thư viện đổi thì tính lại.

Vì sao dùng ở đây: 5 model × cùng 1 kỹ thuật dùng chung đúng 1 tập đã resample — không tính lại SMOTE-ENN 5 lần.

14 Hạ tầng reproducibility & pattern "kind" dispatch

KHÁI NIỆM

Reproducibility: chạy lại cùng code + cùng cấu hình phải ra cùng kết quả.

```
src/config.py#L10-11
```

```
SEED = 42
N_RUNS = 20 # số lần lặp CIES
```

SEED dùng cho benchmark 1 lần (mọi tổ hợp cùng seed 42), Optuna sampler, chọn tập eval và chọn 100 dòng nền SHAP. Trong CIES, run thứ *i* dùng seed *i* cho bootstrap, resampler và model — có chủ đích để 20 lần train lại thực sự khác nhau.

MỨC TÁI LẬP ĐÃ KIỂM CHỨNG

- Cùng máy: chạy lại LR × SMOTE (Sparkov) — 20/20 run trùng tuyệt đối với kết quả đã lưu.
- Khác máy (GPU Kaggle vs CPU local), ANN: mean|SHAP| từng run lệch đáng kể (khác phần cứng → khác thứ tự phép tính số thực → khác trọng số sau train), nhưng CIES gần như trùng (0,843 / 0,838; 0,906 / 0,907) — ANN chỉ tái lập được **về mặt thống kê**.

PATTERN "KIND" — TRÁNH ĐOÁN NGẦM KIỂU MODEL

train.py#L110-126 (model_kind)

```
def model_kind(model):
    if not isinstance(model, dict): return "other"      # sklearn/xgb/catboost
    kind = model.get("kind")
    if kind not in ("lr", "ann"):
        raise ValueError(f"... phải được tạo từ build_model(), Keys hiện có: {...}")
    return kind
```

LR và ANN cần bọc thêm state (scaler, epochs...) nên trả về dict. Trước đây 6 chỗ đoán kiểu qua việc dict có key "scaler" hay không — dễ sai nếu thêm key trùng tên. Nay gắn nhãn "kind" tường minh lúc tạo, dict không đến từ build_model() thì báo lỗi rõ ràng.

Vì sao đáng nhắc: thể hiện tư duy phòng lỗi — lỗi sớm và rõ ràng còn hơn suy luận sai âm thầm.

Nguồn: trích dẫn code lấy trực tiếp từ 11 file source (đọc lại ngày 2026-10-03): config.py, data/preprocess.py, data/encoding.py, imbalance/resamplers.py, models/train.py, models/tune.py, evaluation/metrics.py, explainability/shap_utils.py, explainability/cies.py, utils/isolation.py, utils/jsonio.py; số liệu đối chiếu với reports/study_guide.md, reports/pipeline_report.md, reports/design_decisions.md. Số dòng có thể dịch nếu code thay đổi — khi ôn, tìm theo tên hàm.